DHBW Stuttgart

Karsten Keßler

Datenbanksysteme

1. Einführung

Kurs Informatik TINF25ITH
Stand: 10.09.2026

Inhalte der heutigen Vorlesung

- Begrüßung und Organisatorisches
- Einführung zu Datenbanken
- Erarbeitung des Datenbankbegriffs
- Zugriff
- Vergleich mit Dateien und Excel
- Schichten
- Datenintegration
- Eigenschaften
- Funktionen
- Begriffe

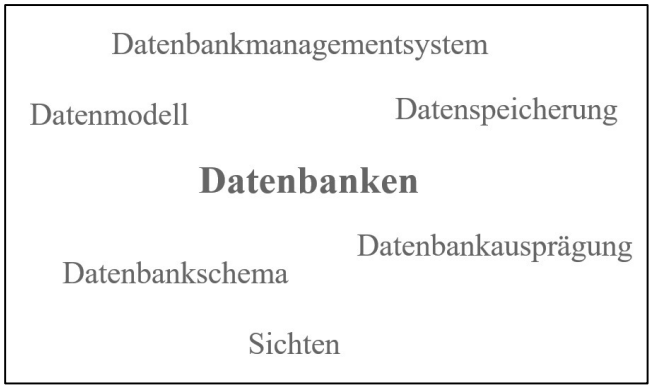
Begrüßung und Organisatorisches



- Rahmen
  Pflichtmodul mit Klausur (3. Semester, 4. Semester) und
  Programmentwurf (4. Semester)
  6 SWS auf zwei Semester verteilt
- Kontaktdaten
  karsten.kessler@lehre.dhbw-stuttgart.de
  Fragen gerne direkt in oder nach der Vorlesung
- Moodle
  Folien (pdf)
  Übungen (Aufgabenstellungen etc.)
- Begleitendes Repository: https://github.com/karkessler/dhbw-db-2627

Modulhandbuch Informatik zur Vorlesung Datenbanken

- Grundkonzepte und Datenmodellierung (u.a Entity Relationship Modell)
- Relationales Datenmodell
- Normalformen
- Relationaler Datenbankentwurf
- Mehrbenutzerbetrieb und Transaktionskonzepte
- Architekturen von Datenbanksystemen



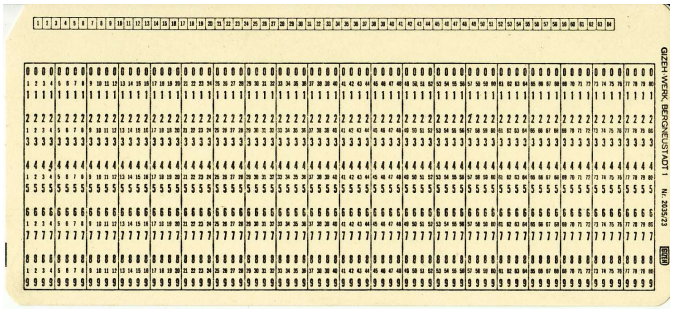
Prüfungsleistung

- 3. Semester Klausur, 1h (40%)
- 4. Semester: benotete Projektarbeit + Fachgespräch (60%), max. 3 Personen

Übersicht der Gesamtvorlesung (vorläufig)

Sem.	Nr.	Thema	Inhalte (Auszug)
3	1	Einführung	Einführung, Begriffe, Eigenschaften, Funktionen, Datenintegration
	2	Datenbankmodelle u. -architekturen	Architektur und SPARC, verschiedene Datenbankmodelle, Entwicklung
	3	ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
	4	Relationales Modell	Relationales Datenbankmodell im Detail, Schlüssel und Integritätsbedingungen, Abbildung ER → Relation
	5	Relationale Algebra	Relationale Algebren, Operatoren u. Operatorbäume, Anwendung, Nützlichkeit
	6	Normalformen	Anomalien, funktionale Abhängigkeiten, 1NF-3NF und BCNF, Dekomposition, Synthesealgorithmus
4	7	NoSQL & moderne Datenmodelle	Dokument-, Graph- und Vektordatenbanken, Einsatzszenarien
	8	Transaktionen	ACID, grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
	9	SQL DML	Datenmanipulation mit SQL
	10	SQL DDL	Datendefinition mit SQL
	11	Integrität	Integrität im relationalen Modell, Umsetzung (in SQL)
	12	Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
	13	Indizes	Interne Strukturen (B-Bäume, Hashing), Nützlichkeit für Anfragen

Vergleich von Dateien mit Datenbanken



Hollerith-Lochkarte: 80 Spalten, festes Satzformat, jedes Feld belegt einen fest definierten Spaltenbereich.

Auch Fortran/COBOL nutzten dieses Raster: 80 Spalten je Zeile, 73 bis 80 nur Kartennummer.

Beispiel für ein Dateischema:

Name (6 Zeichen)	Vorname (7 Zeichen)	PLZ (5 Zeichen)
Timˍˍˍ	Klausˍˍ	10154
Dorisˍ	Ortusˍˍ	74394

Daten werden dabei in einem festen Schema organisiert, um Konsistenz und Vergleichbarkeit zu gewährleisten.

Vergleich von Dateien mit Datenbanken

Dateien	Datenbanken
Dateiformat vom Programmierer vorgegeben, individuell verschieden	Einheitliches Schema, zentral im DBMS beschrieben
Formatänderungen ziehen große Aufwände nach sich	Schemaänderungen ohne Anpassung aller Programme
Daten mehrfach und redundant gespeichert, Konsistenz schwer zu halten	Redundanzfreie Datenhaltung, Konsistenz wird erzwungen
Wenig Möglichkeiten zur synchronen Behandlung, keine Transaktionen	Transaktionen und Mehrbenutzerbetrieb
Eingeschränkte Abfragemöglichkeiten, Skalierungsgrenzen	Mächtige Abfragesprache (SQL), skaliert mit Datenmenge
Wenig Schutz vor Datenverlust, aufwändige Sicherung	Backup und Recovery im DBMS enthalten

Wozu Datenbanken?

- → **Zu viel Informationen** sind genauso nutzlos wie zu wenig Informationen.
- → Entscheidend ist, die **richtigen Informationen** zu finden.
- → Dabei sollten unternehmensrelevante Daten **strukturiert** in Datenbanken abgelegt werden.
- → **Datenintegrität**: Datenbanken bieten Mechanismen zur Sicherstellung der Datenintegrität, sodass die Daten korrekt, konsistent und zuverlässig bleiben.
- → **Datensicherheit**: Durch Zugangskontroll- und Authentifizierungsfunktionen können Administratoren den Zugang zu sensiblen Daten einschränken, was die Datensicherheit erhöht.
- → **Effiziente Datenorganisation**: Datenbanken ermöglichen eine strukturierte und organisierte Speicherung großer Datenmengen, was ein effizientes Abrufen und Verwalten der Daten erleichtert.
- → **Gleichzeitiger Zugriff**: Datenbanken ermöglichen den gleichzeitigen Zugriff durch mehrere Benutzer oder Anwendungen, ohne die Datenkonsistenz zu beeinträchtigen.
- → **Skalierbarkeit**: Datenbanken können skaliert werden, um große Datenmengen und hohe Benutzerlasten zu bewältigen.

Vergleich von Excel mit Datenbanken:

Da Tabellenkalkulationsprogramme oft nur aus einer einzigen Tabelle bestehen, können redundante Daten entstehen. Daten werden mehrmals wiederholt, bspw. stehen die MA-Daten in jedem Projekt, dem der MA angehört.

Auszug aus einer Excel-Liste „Projekte“:

Projekt	Projektleiter	Mitarbeiter	Abteilung	Telefon
Portal-Relaunch	Meier, Anna	Schmidt, Jan	IT	0711-123
Portal-Relaunch	Meier, Anna	Weber, Lea	Vertrieb	0711-456
CRM-Einführung	Weber, Lea	Weber, Lea	Vertrieb	0711-456
Datenmigration	Meier, Anna	Schmidt, Jan	IT	0711-123

Gegenüberstellung: eine einzige Tabelle vs. mehrere verknüpfte Tabellen

Kriterium	Eine einzige Tabelle	Mehrere verknüpfte Tabellen
Redundanz	Dieselben Angaben wiederholen sich in vielen Zeilen	Jede Information wird genau einmal gespeichert
Konsistenz	Eine Änderung muss überall nachgezogen werden	Eine Änderung an genau einer Stelle genügt
Beziehungen	Nur implizit, über die Anordnung der Spalten	Explizit, über Schlüssel zwischen den Tabellen
Integrität	Das Programm prüft die Eingaben nicht	Das DBMS erzwingt die hinterlegten Regeln
Mehrbenutzer	Datei ist gesperrt oder der letzte Stand gewinnt	Gleichzeitiger Zugriff wird koordiniert

Fachbegriffe: typische Probleme bei redundanter Speicherung

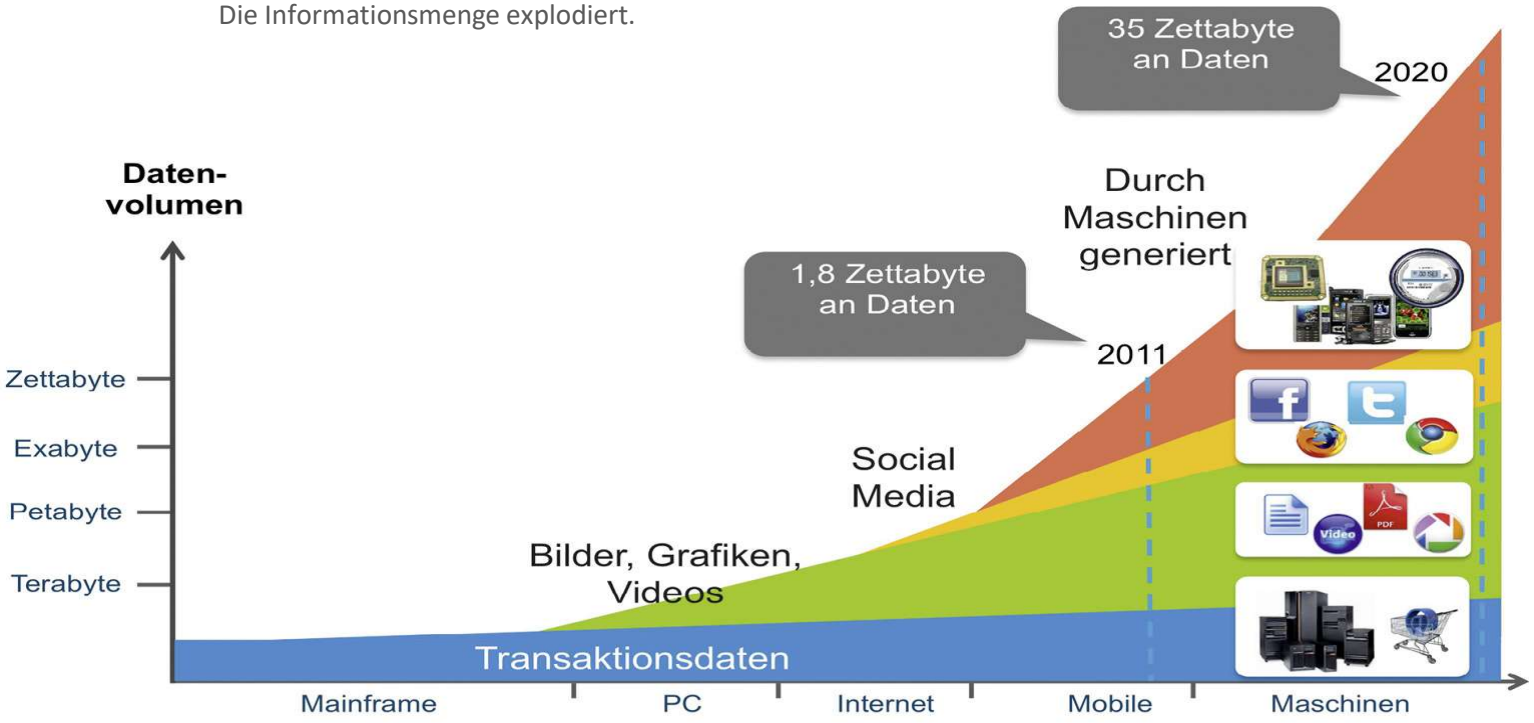
Einfügeanomalie
Ein Objekt lässt sich nicht erfassen, weil in derselben Zeile Angaben zu einem anderen Objekt fehlen, die es noch nicht gibt.

Änderungsanomalie
Eine einzige inhaltliche Änderung muss in vielen Zeilen wiederholt werden. Wird eine übersehen, widersprechen sich die Daten.

Löschanomalie
Beim Löschen eines Datensatzes verschwinden zugleich Informationen, die inhaltlich davon unabhängig sind.

→ *Vertiefung im Übungsblatt 1, Aufgabe 1.5*

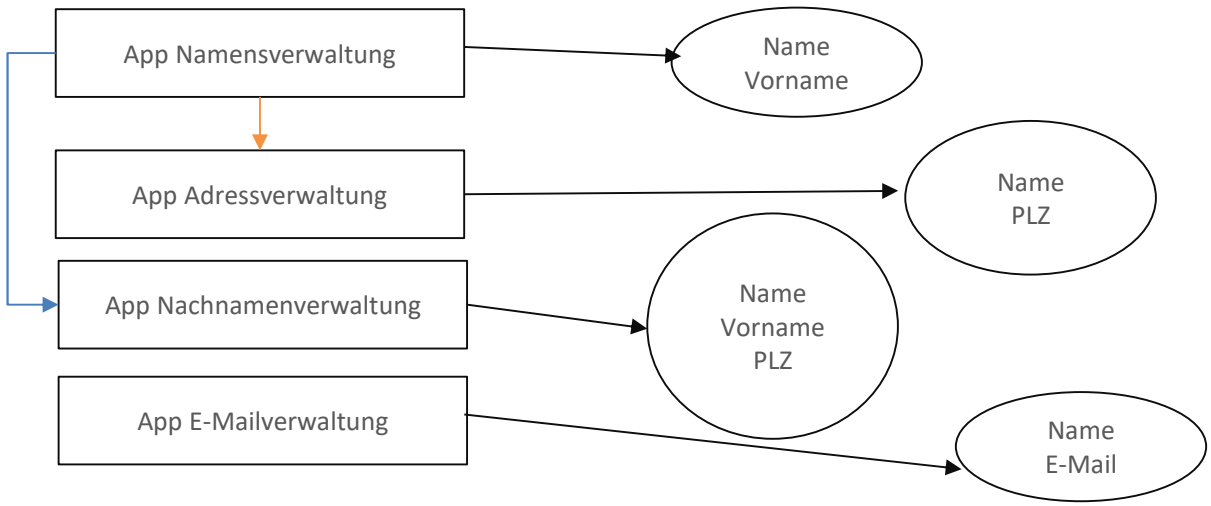
Die Informationsmenge explodiert.



https://www.digitalbusiness-cloud.de/richtig-entscheiden-mit-big-data/

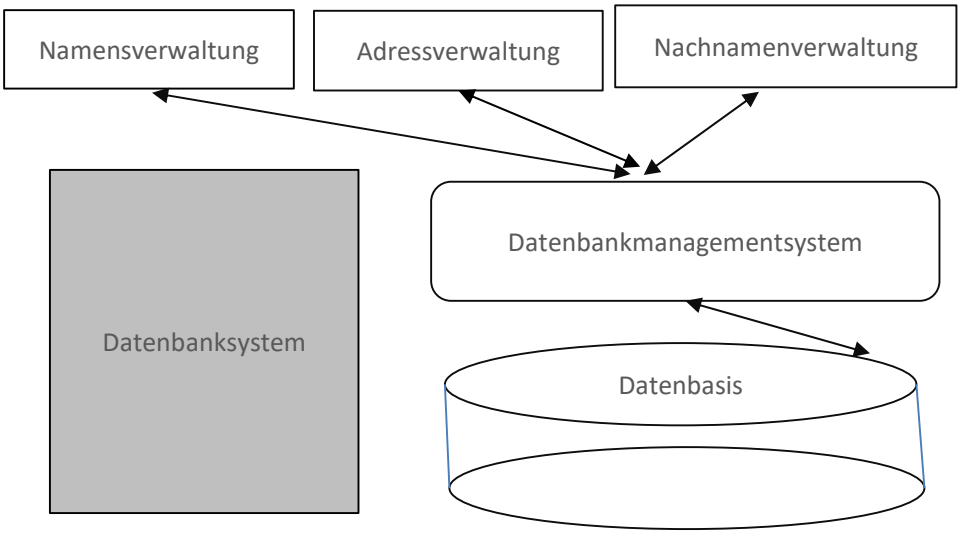
Beispiel: Adressverwaltung auf Dateibasis

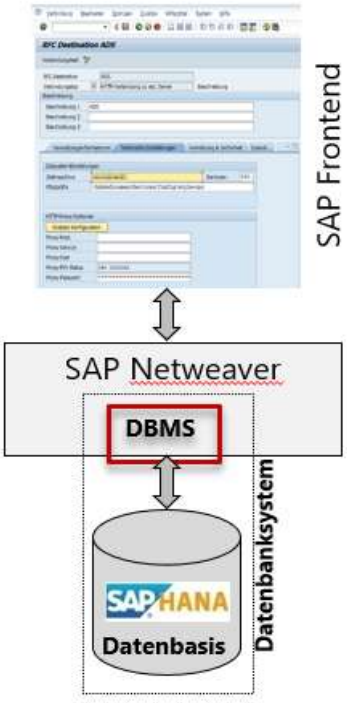
→ Ändert sich der Name in einer App, müssen andere Dateien angepasst werden.



→ Das Zusammenführen von Daten aus verschiedenen Dateien kann komplex und fehleranfällig sein, insbesondere wenn die Dateien unterschiedliche Formate oder Strukturen haben. Dies kann zu Inkonsistenzen und Datenverlust führen.

Beispiel: Adressverwaltung mit Datenbanksystem





Die Vermeidung von *Redundanz* (mehrfach gehaltene Daten) und daraus resultierender *Inkonsistenz* (mehrfach und dabei unterschiedlich gehaltene Daten) ist ein *Hauptgrund* für den Einsatz von DBMS.

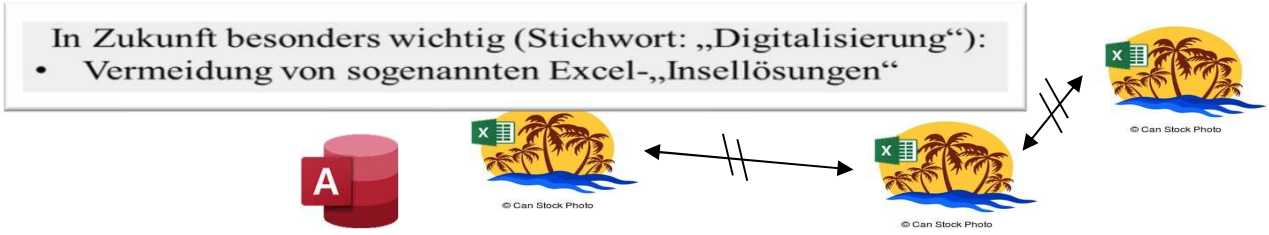
Primäre Aufgaben des Datenbanksystems:

- Beschreibung
- Speicherung und Pflege
- Wiedergewinnung

... (sehr großer) Datenmengen, die von verschiedenen Programmen genutzt werden.

- Sicherstellung der Datenintegrität
- Zugriffskontrolle und Berechtigungsverwaltung:
- Datensynchronisation und -aktualisierung

In Zukunft besonders wichtig (Stichwort: „Digitalisierung“):
- Vermeidung von sogenannten Excel-„Insellösungen“





Vergleich von Datenbanken mit Bibliotheken





- **Was macht eine gute Bibliothek aus?**

  → Die vorhandene Bücherauswahl
  Eine gute Bibliothek bietet eine breite und vielfältige Auswahl an Büchern, die verschiedene Interessen und Bedürfnisse der Benutzer abdeckt. Dies umfasst sowohl aktuelle Bestseller als auch klassische Werke und spezialisierte Fachliteratur.

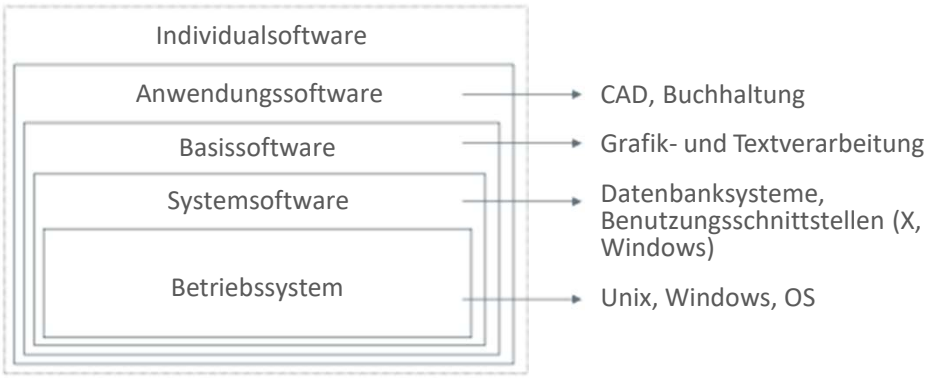
  → Dass ich das gesuchte Buch auch finde
  Eine effiziente Katalogisierung und Organisation der Bücher ist entscheidend. Dies bedeutet, dass Bücher nach Themen, Autoren oder anderen relevanten Kriterien sortiert sind, sodass Benutzer schnell und einfach das gewünschte Buch finden können. Ein gut gepflegtes Suchsystem, wie ein Online-Katalog, unterstützt dies zusätzlich.

- **Was bedeutet das für eine Datenbank?**

  → Datenspeicher bzw. sein Inhalt
  Eine gute Datenbank speichert eine große Menge an Daten, die für verschiedene Anwendungen und Benutzer relevant sind. Dies kann strukturierte Daten wie Tabellen und Datensätze oder unstrukturierte Daten wie Dokumente und Multimedia-Dateien umfassen. Die Qualität und Relevanz der gespeicherten Daten sind entscheidend für die Nützlichkeit der Datenbank.

  → Effiziente Zugriffsstrukturen
  Effiziente Zugriffsstrukturen in einer Datenbank ermöglichen es Benutzern, schnell und einfach auf die benötigten Daten zuzugreifen.

Datenbanken und Softwareschichten



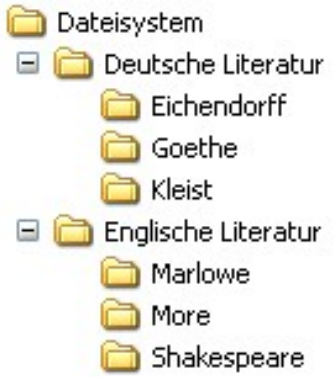
- Computersysteme werden in Schichten aufgeteilt
- Softwarepakete können mehrere solcher Schichten umfassen (z.B. ausführbare Dateien, Bibliotheken, Konfigurationsdateien, Skripte, Meta-Informationen)

Quelle: Saake, G.; Sattler, K.-U.; Heuer, A.: Datenbanken, Konzepte und Sprachen, Abb. 1.1 „Aufteilung in Softwareschichten“.

Eine einfache Organisationsform: das *Dateisystem*



Die leeren Regale sind aufgestellt ...



Das Dateisystem ist eingerichtet ...

Datenbanken Einleitung

Ohne Datenbanken…

- Basis- und Anwendungssoftware verwaltet ihre eigenen Daten in eigenen Dateien
- Textverarbeitung verwaltet Texte, Artikel und Adressen
- Buchhaltung speichert Artikel- und Adressinformationen
- Lagerverwaltung benötigt Artikel und Aufträge
- Auftragsverwaltung benötigt und verändert Aufträge, Artikel und Kundenadressen

- Redundanz
- Verschwendung von Speicherplatz
- Vergessen von lokalen Änderungen
- Inkonsistenzen

Datenintegration

- Effizienz der Datenhaltung
- Störungsfreier paralleler Zugriff auf Daten
- Fehlende Datenunabhängigkeit
- Zugriffskontrolle / Datensicherheit

- => Anwendungs-programmierer muss die interne Darstellung kennen

Datenintegration

Mit Datenbanksystemen: Basis- und Anwendungssoftware arbeitet auf denselben Daten

Eigenschaften

- Anfragesprachen *Abstraktion / Innere Optimierung*
  Sprachen, die verwendet werden,
  um Datenbankabfragen zu formulieren.
- Transaktionskonzept
  Mechanismen, die sicherstellen, dass Datenbanktransaktionen korrekt und sicher ausgeführt werden.
- Drei-Ebenen-Architektur *Anwendungsschicht / logische Sicht / interne Datenstrukturen*
  Ein Modell, das die Datenbankarchitektur
  in drei Ebenen unterteilt (Anwendungsschicht, logische Sicht, interne Datenstrukturen).
- Zugriffskontrolle / Datensicherheit
  Maßnahmen zum Schutz der Daten vor unbefugtem Zugriff und zur Sicherstellung der Datenintegrität.

Logisches Datenbankschema
- Welche Tabellen oder welche Objekte werden gespeichert?
- Welche Attribute (Spaltenüberschriften) beschreiben die Einträge?

Prinzipien und Aufgaben: Basisfunktionalität nach Codd (1982)

	Funktionalität	Beschreibung
1	Integration	Einheitliche Verwaltung aller von Anwendungen benötigten Daten
2	Operationen	Operationen zum Speichern, Suchen und Ändern müssen vorhanden sein
3	Katalog	„Data Dictionary“ ermöglicht Zugriffe auf die Beschreibung der Daten
4	Benutzersichten	Unterschiedliche Anwendungen mit unterschiedlicher Sicht auf Daten
5	Konsistenzüberwachung	Das DBMS kontrolliert die Korrektheit der Daten bei Änderungen
6	Zugriffskontrolle	Datenschutz, unautorisierte Zugriffe verhindern
7	Transaktionen	Zusammenfassung von DB-Änderungen zu Funktionseinheiten, die als Ganzes ausgeführt werden
8	Synchronisation	u.a. Auflösen von Schreibkonflikten für verschiedene Benutzer
9	Datensicherung	Wiederherstellung von Daten nach Systemfehlern

Datenbanken bzw. Datenbanksysteme bestehen aus:
- den gespeicherten Daten (Datenbasis)
- den Programmen zum Zugriff auf die Daten (Datenbankverwaltungssystem oder Database Management System, DBMS)

• Datenbasis (DB)	Strukturierter Datenbestand
• Datenbankmanagementsystem (DBMS)	Software zur Verwaltung einer Datenbank
• Datenbanksystem (DBS)	DB und DBMS zusammen

Datenbanksystem:

Ein Datenbanksystem umfasst sowohl die Datenbank selbst als auch das Datenbankmanagementsystem (DBMS). Es besteht aus der Hardware, der Software und den Daten, die zusammenarbeiten, um die Speicherung, Verwaltung und Abfrage von Daten zu ermöglichen. Das Datenbanksystem sorgt dafür, dass die Daten sicher und effizient verarbeitet werden können.

Datenbasis:

Die Datenbasis ist der eigentliche Speicherort der Daten innerhalb des Datenbanksystems. Sie enthält alle gespeicherten Daten in einer strukturierten Form, wie Tabellen, Datensätze und Beziehungen zwischen den Daten. Die Datenbasis ist das Fundament, auf dem das gesamte Datenbanksystem aufbaut.

Datenbankmanagementsystem:

Das Datenbankmanagementsystem ist die Software, die die Verwaltung der Datenbasis übernimmt. Es ermöglicht Benutzern und Anwendungen den Zugriff auf die Daten, stellt sicher, dass die Daten konsistent und sicher sind, und optimiert die Leistung der Datenbank. Das DBMS bietet Funktionen wie Datenabfrage, Datenmanipulation, Datensicherheit und Backup.

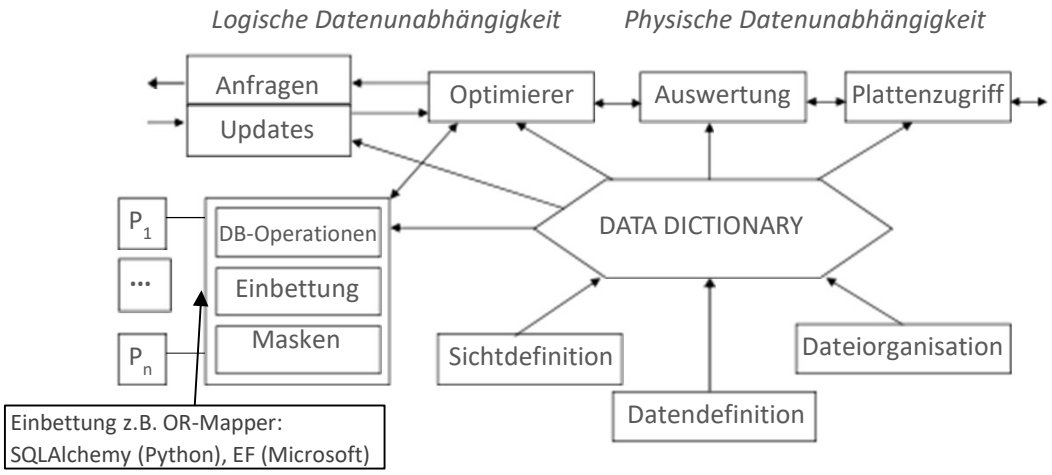
Architektur eines DBMS

Externe Ebene **Konzeptuelle Ebene** **Interne Ebene**

Hier werden Anfragen und Updates von Benutzern verarbeitet

Diese Ebene umfasst den Optimierer und die Auswertung, die für die Verarbeitung und Optimierung der Anfragen zuständig sind.

Hier findet der Plattenzugriff statt, und es gibt Komponenten wie das DATA DICTIONARY und die DB-Operationen.



Architektur eines DBMS

• **Konzeptuelle Ebene**

Logische Gesamtsicht aller Daten unabhängig von den einzelnen Anwendungsprogrammen

- Ergebnis des Datenbankentwurfs
- Beschreibung der Objekttypen und Relationen
- Keine Details zur Speicherung
- Spezifiziert mit DDL (Data Definition Language)

• **Externe Ebene**

Zusammenfassung aller Views

- Abkopplung von der Datenbank von Änderungen und Erweiterungen an Anwenderschnittstellen (logische Datenunabhängigkeit)

• **Interne Ebene**

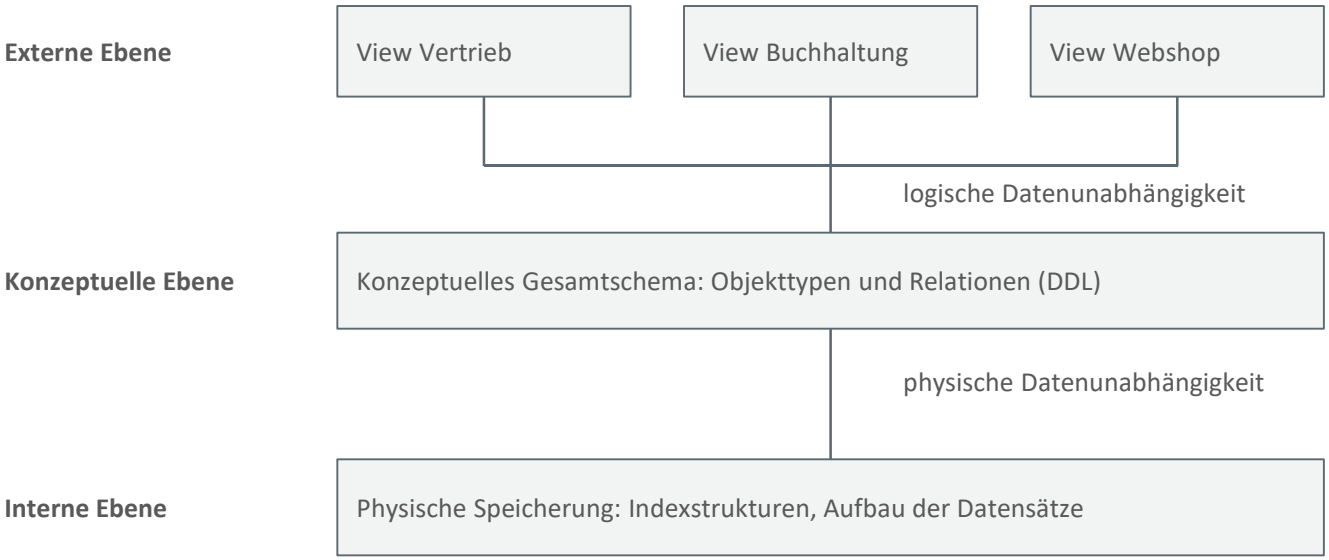
Physische Speicherung der Daten:

Indexstrukturen

Aufbau der gespeicherten Datensätze

- Unabhängigkeit der Anwendungen von Änderungen am internen Schema (physische Datenunabhängigkeit)

Drei-Ebenen-Architektur eines DBMS



- **Data Dictionary**

Ein zentraler Katalog, der Metadaten enthält. Diese Metadaten beschreiben die Struktur, die Definitionen und die Darstellungsregeln aller Anwendungsdaten in einem Unternehmen.

- **Definition und Struktur**: Das Data Dictionary enthält Informationen über Tabellen, Felder, Datentypen, Beziehungen zwischen Datenobjekten und andere strukturelle Details
  Es hilft dabei, den Datenbestand einheitlich und redundanzfrei zu halten

- **Funktion**: Es dient als Nachschlagewerk für Entwickler, Datenbankadministratoren und andere IT-Fachleute.
  Es ermöglicht eine konsistente Datenmodellierung und erleichtert die Verwaltung und Pflege der Datenbank

- **Aktive und passive Data Dictionaries**:
  Aktiv: Reflektiert immer den aktuellen Stand des Datenmodells und wird automatisch aktualisiert.
  Passiv: Muss manuell aktualisiert werden, wenn Änderungen an der Datenbankstruktur vorgenommen werden.

- **Ebenen der Datenmodellierung**:
  Ein Data Dictionary kann auf verschiedenen Modellierungsebenen verwendet werden, wie der konzeptuellen, logischen und physischen Ebene

Ein Beispiel für den Inhalt eines Data Dictionary könnte so aussehen:

Tabelle: KURS
- Feld: KNR (Datentyp: Integer, Primärschlüssel)
- Feld: KURSBEZEICHNUNG (Datentyp: Varchar)
- Feld: KURSORT (Datentyp: Varchar), Feld: TNR (Integer, Fremdschlüssel auf THEMA)

Data Dictionary in MySQL (Auszug aus information_schema)

Nur lesender Zugriff, Änderungen nur über DDL. Alle Abfragen sind voll qualifiziert und ohne USE lauffähig.

View (information_schema)	Inhalt	Beispielabfrage
TABLES	Metadaten aller Tabellen und Views: Name, Schema, Engine, Zeilenzahl	`SELECT table_name, engine FROM information_schema.tables WHERE table_schema='Vorlesungen';`
COLUMNS	Spalten je Tabelle: Datentyp, Default-Wert, NULL-Zulässigkeit, Position	`SELECT column_name, data_type FROM information_schema.columns WHERE table_schema='Vorlesungen' AND table_name='KURS';`
KEY_COLUMN_USAGE	Zuordnung von Primär- und Fremdschlüsseln zu Spalten	`SELECT column_name, referenced_table_name FROM information_schema.key_column_usage WHERE table_schema='Vorlesungen' AND table_name='KURS';`
STATISTICS	Indexe mit ihren Spalten und deren Kardinalität	`SELECT index_name, cardinality FROM information_schema.statistics WHERE table_schema='Vorlesungen' AND table_name='KURS';`
SCHEMATA	Alle Datenbanken (Schemata) mit Zeichensatz und Kollation	`SELECT schema_name, default_character_set_name FROM information_schema.schemata WHERE schema_name='Vorlesungen';`
VIEWS	Definition (SELECT-Anweisung) und Aktualisierbarkeit aller Views	`SELECT table_name, view_definition FROM information_schema.views WHERE table_schema='Vorlesungen';`
TABLE_CONSTRAINTS	Constraints je Tabelle: PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK	`SELECT constraint_name, constraint_type FROM information_schema.table_constraints WHERE table_schema='Vorlesungen';`
REFERENTIAL_CONSTRAINTS	Fremdschlüsselbeziehungen mit ON UPDATE-/ON DELETE-Regeln	`SELECT constraint_name, delete_rule FROM information_schema.referential_constraints WHERE constraint_schema='Vorlesungen';`
ROUTINES	Gespeicherte Prozeduren und Funktionen inklusive Quellcode	`SELECT routine_name, routine_type FROM information_schema.routines WHERE routine_schema='Vorlesungen';`
TRIGGERS	Trigger mit Ereignis, Zeitpunkt und Aktionsanweisung	`SELECT trigger_name, event_manipulation FROM information_schema.triggers WHERE trigger_schema='Vorlesungen';`
ENGINES	Verfügbare Storage Engines: Speicherverfahren je Tabelle, bestimmt Transaktionen, Sperren und Fremdschlüssel	`SELECT engine, support, transactions FROM information_schema.engines; SHOW ENGINES; ALTER TABLE KURS ENGINE=InnoDB;`

Literaturempfehlung

Kemper, Alfons; Eickler, André (2015): Datenbanksysteme. Eine Einführung. 10., aktualisierte und erw. Aufl. Berlin, Boston Mass.: De Gruyter Oldenbourg (De-Gruyter-Oldenbourg-Studium).

Saake, Gunter; Sattler, Kai-Uwe; Heuer, Andreas (2018): Datenbanken. Konzepte und Sprachen. 6., überarbeitete Aufl. Frechen: mitp (mitp Professional).

Vielen Dank für Ihre Aufmerksamkeit!